Integration patterns, drawn

55 patterns on one page: how systems share data, how they are wired, and how messages are built, routed, transformed and protected. Messaging names follow Hohpe and Woolf's Enterprise Integration Patterns.

system or step queue or topic message changed or reply message failure

If you need...

An answer right now	Request–reply, remote call (API)
Tell many systems something happened	Publish–subscribe, event message
Systems use different formats	Message translator, canonical data model, normalizer
Spread work across workers	Competing consumers, splitter
Combine results from several calls	Scatter–gather, aggregator
Survive downstream outages	Guaranteed delivery, retry with backoff, circuit breaker, dead letter channel
Multi-step process across services	Process manager, saga, routing slip
Replace a legacy system gradually	Strangler fig, anti-corruption layer
Publish an event and save data safely	Transactional outbox, change data capture

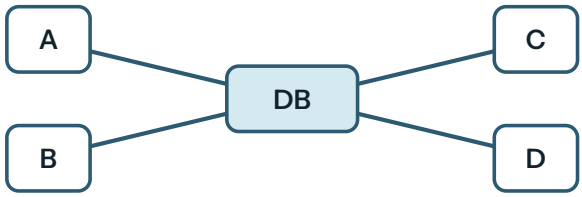
Integration styles

The four basic ways applications share data.



File transfer

Export a file, drop it, import it. Simple and batch-friendly, but slow to react.



Shared database

Apps read and write the same tables. Tight coupling; the schema becomes the contract.



Remote call (RPC / API)

Caller waits for the answer. Immediate, but both sides must be up at once.

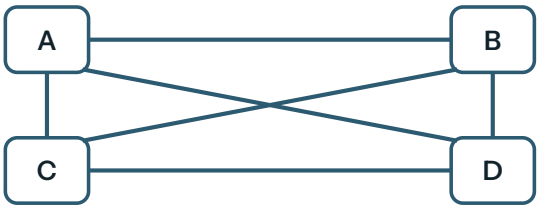


Messaging

Send through a queue. Sender and receiver need not be available at the same moment.

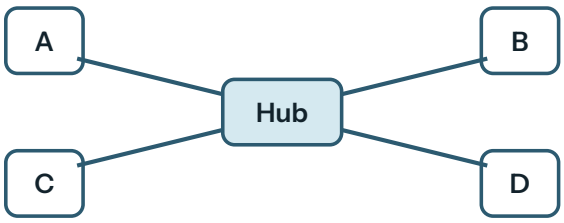
Topologies

How many systems get wired together.



Point-to-point

Each pair wired directly. Fine for a few apps; links grow as n(n−1)/2.



Hub and spoke

One broker in the middle routes and transforms. One place to govern, one place to fail.







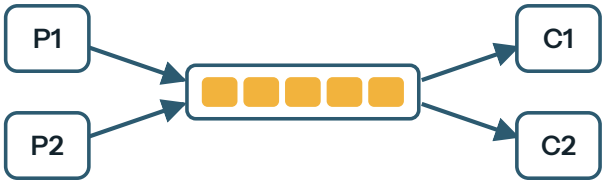
Message bus / ESB

Shared backbone with a common format. Apps plug in through adapters.



API-led connectivity

System APIs wrap sources, process APIs orchestrate, experience APIs serve channels.



Event streaming backbone

Producers append to a durable log. Consumers read at their own pace and can replay.

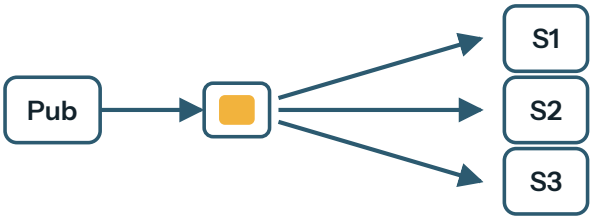
Messaging channels

Where messages travel and what happens when delivery goes wrong.



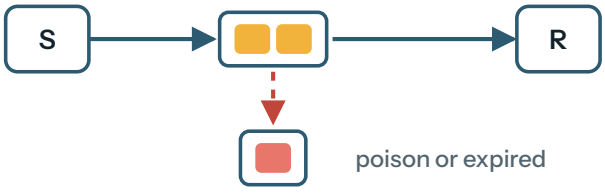
Point-to-point channel

Many senders, but exactly one receiver gets each message.



Publish–subscribe channel

Every subscriber gets its own copy of each event.



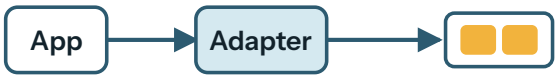


Dead letter channel

Messages that keep failing go to a side queue instead of blocking the line.

Guaranteed delivery

Persist before sending; keep it until the receiver acknowledges.



Channel adapter

Connects an app that doesn't speak messaging (database, file, SaaS) to a channel.



Messaging bridge

Links two messaging systems so apps on each side can talk.

Message construction

What a message means and how replies find their way back.



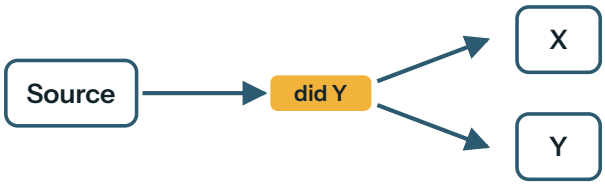
Command message

Tells the receiver to do something. One intended handler.



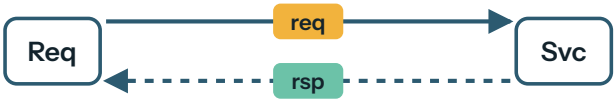
Document message

Carries data for the receiver to use, with no instruction attached.



Event message

Announces that something happened. The sender doesn't know who listens.



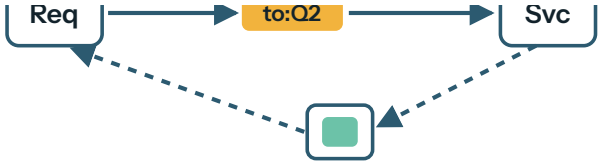
Request–reply

Two channels: one carries the request, one carries the answer.



Correlation ID

The reply carries the request's ID so the caller can match them up.



Return address

The request says where the reply should be sent.

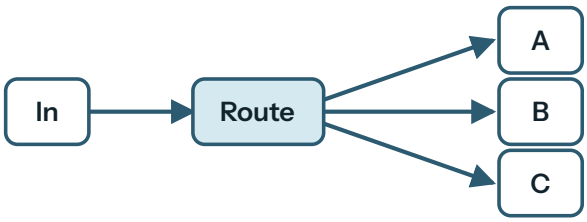


Message sequence

Big payloads are split into numbered parts so they can be rebuilt.

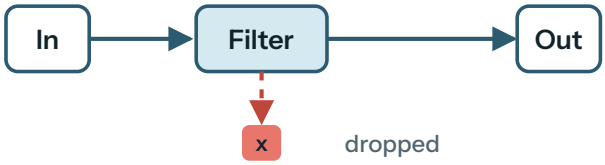
Routing

Deciding where a message goes, and combining or splitting the flow.



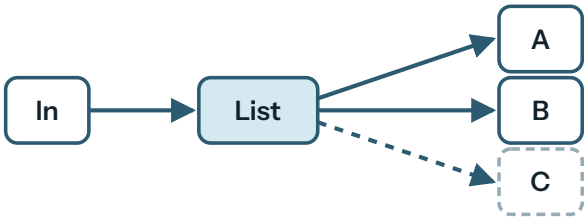
Content-based router

Looks inside the message and picks the destination.

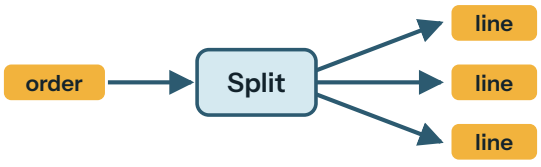


Message filter

Drops messages the receiver doesn't care about.



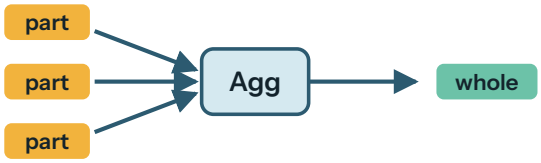
Recipient list



Splitter

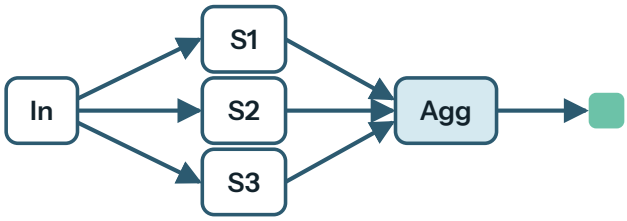
Sends one message to a computed list of receivers.

Breaks one message into several, one per item.



Aggregator

Collects related messages and emits one combined result.



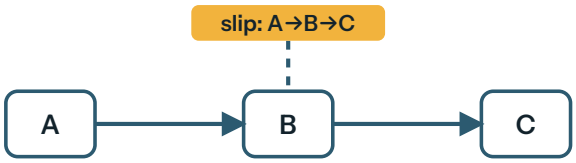
Scatter–gather

Sends a request to several services in parallel, then merges the replies.



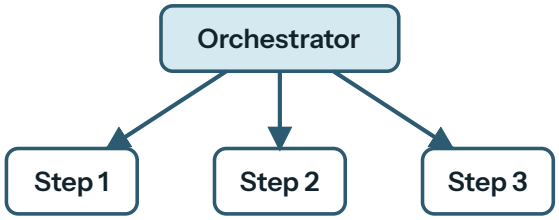
Resequencer

Puts out-of-order messages back in order.



Routing slip

The message carries its own route; each step passes it to the next.

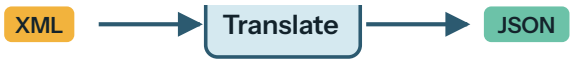


Process manager

A central orchestrator drives each step and decides what comes next.

Transformation

Changing the shape or size of a message on its way through.



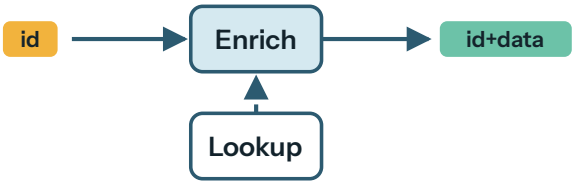
Message translator

Converts one format or schema into another.



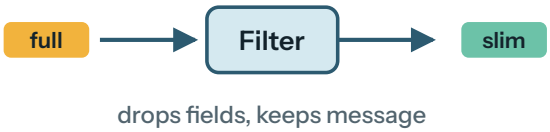
Envelope wrapper

Wraps a message in the headers a transport needs, then unwraps it at the end.



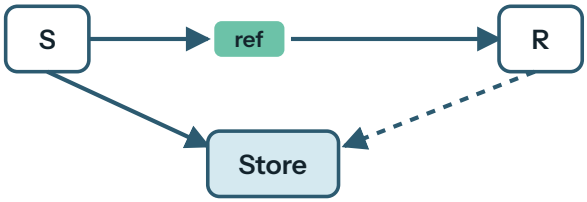
Content enricher

Adds missing data from another source.



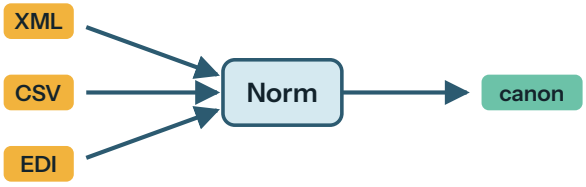
Content filter

Removes fields and keeps a smaller message. The message itself still flows on.



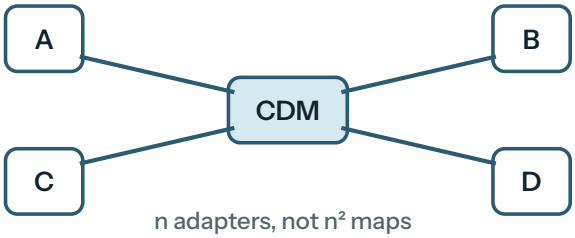
Claim check

Stores the big payload aside and passes a small reference instead.



Normalizer

Turns many incoming formats into one standard shape.



Canonical data model

One shared format in the middle: n adapters, not n² translations.

Endpoints

How applications take messages in and hand them to real work.



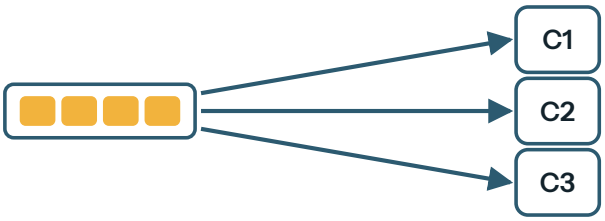
Polling consumer

Asks the queue on a schedule: is there anything for me?



Event-driven consumer

The channel pushes each message to the handler as it arrives.



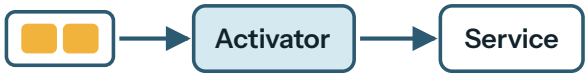
Competing consumers

Several workers pull from one queue. Each message goes to exactly one.



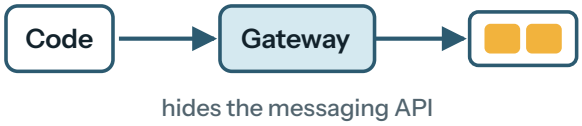
Idempotent receiver

Handles a duplicate delivery as if it were one, so retries are safe.



Service activator

Turns an incoming message into a call on a service.



Messaging gateway

Wraps the messaging API so application code never touches it.

Data movement

Moving or exposing data between stores.



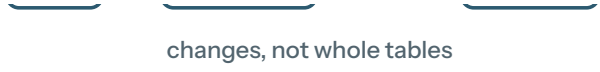




scheduled, bulk

Batch ETL

Extract, transform and load on a schedule. Bulk and predictable, not real-time.



changes, not whole tables

Change data capture

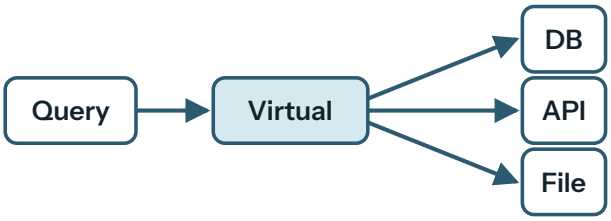
Reads the database log and streams only what changed.



needs conflict rules

Bidirectional sync

Two systems keep each other up to date. Needs rules for conflicts.



Data federation

A virtual layer queries many sources in place, without copying data.

Resilience and evolution

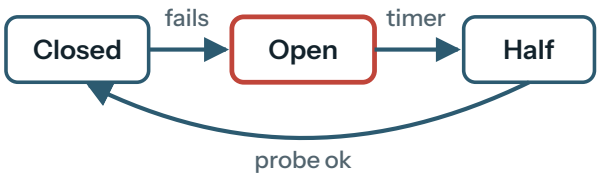
Surviving failure and replacing systems without a big bang.



wait doubles each time

Retry with backoff

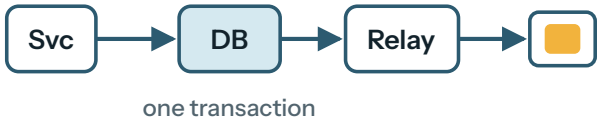
Try again after a growing pause so a struggling service can recover.



probe ok

Circuit breaker

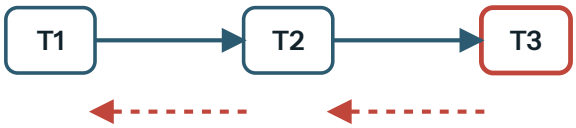
Stops calling a failing service for a while, then probes before resuming.



one transaction

Transactional outbox

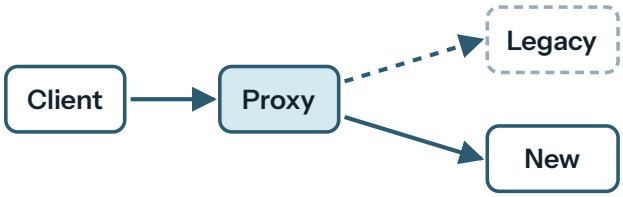
Write the event and the data in one transaction; a relay publishes it afterwards.



compensate in reverse

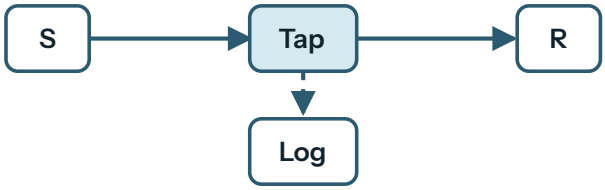
Saga

A chain of local steps. If one fails, earlier steps are undone by compensations.



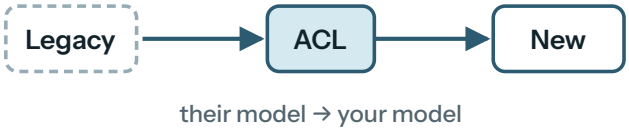
Strangler fig

Route traffic through a proxy and move features to the new system one by one.



Wire tap

Copies messages to a side channel for logging or audit without disturbing the flow.



Anti-corruption layer

Translates a legacy model into yours so its quirks don't leak into the new design.

Dashed arrows are optional, deferred or error paths. Dimmed dashed boxes are systems you are moving away from or that are not selected.